Contents

1. Project Title and Team Members 2
2. Revisions from Previous Deliverable 3
3. Model Evaluation Metrics 4
4. Initial Model Experimentation 5
5. Selection of Best-Performing Models and Pre-processed Datasets 8
6. Pipeline Definition 9
7. Hyperparameter Tuning 11
8. Final model evaluation 14
   8.1 Evaluation at Default Threshold 14
   8.2 Threshold Tuning 15
   8.3 Weighted Voting 17
   8.4 Extended Experiments and Final Model Comparison 18
9. Discussion 20
   9.1 Key Findings 20
   9.2 Comparison with Literature 21
   9.3 Domain-Specific Validation 21
   9.4 Study Limitations 22
10. References 23

1. Project Title and Team Members

Project title: Predicting Stroke Risk from Demographic, Lifestyle, and Clinical Factors

Role / Background	**Name**	**Student ID**
Computer Science	Λουκία Σιήκκη	UC1066315
Computer Science	Στέφανος Παντελή	UC1065916
Computer Science	Ραφαήλ Μυτιληναίος	UC1066383

2. Revisions from Previous Deliverable

- The number of preprocessed dataset versions was expanded from 8 to 16. In Deliverable 2, we combined two outlier transformations (Logarithmic, Yeo-Johnson) with two BMI imputation strategies (Mean, KNN) and then duplicated each of the above versions with SMOTE to produce 8 versions in total. In this deliverable, each of those 8 versions was additionally duplicated with *RobustScaler* applied as a final step, producing a scaled counterpart for every version. This allows a direct comparison of whether scaling benefits each model-preprocessing combination.

- SVD dimensionality reduction (*TruncatedSVD*) was implemented and incorporated as an optional pipeline step. This was listed as a future development in Deliverable 2 and has now been tested both in the initial experimentation phase and as a dedicated pipeline.

- All preprocessing steps were refactored into sklearn-compatible transformer classes by inheriting from *BaseEstimator* and *TransformerMixin*. This allowed all steps (cleaning, transformation, imputation, SMOTE, BMI class encoding, and scaling) to be embedded directly inside *Pipeline* objects.

Acronym of dataset version	**Full Description**
lg_kn	Logarithmic Transformation + KNN BMI Imputer
lg_kn_sc	Logarithmic Transformation + KNN BMI Imputer + RobustScaler
lg_kn_sm	Logarithmic Transformation + KNN BMI Imputer + SMOTE
lg_kn_sm_sc	Logarithmic Transformation + KNN BMI Imputer + SMOTE + RobustScaler
lg_me	Logarithmic Transformation + Mean BMI Imputer
lg_me_sc	Logarithmic Transformation + Mean BMI Imputer + RobustScaler
lg_me_sm	Logarithmic Transformation + Mean BMI Imputer + SMOTE
lg_me_sm_sc	Logarithmic Transformation + Mean BMI Imputer + SMOTE + RobustScaler
yj_kn	Yeo-Johnson Transformation + KNN BMI Imputer
yj_kn_sc	Yeo-Johnson Transformation + KNN BMI Imputer + RobustScaler
yj_kn_sm	Yeo-Johnson Transformation + KNN BMI Imputer + SMOTE
yj_kn_sm_sc	Yeo-Johnson Transformation + KNN BMI Imputer + SMOTE + RobustScaler
yj_me	Yeo-Johnson Transformation + Mean BMI Imputer
yj_me_sc	Yeo-Johnson Transformation + Mean BMI Imputer + RobustScaler
yj_me_sm	Yeo-Johnson Transformation + Mean BMI Imputer + SMOTE
yj_me_sm_sc	Yeo-Johnson Transformation + Mean BMI Imputer + SMOTE + RobustScaler

lg = Logarithmic
yj = Yeo-Johnson
kn = KNN imputer
me = Mean imputer
sm = SMOTE applied
sc = RobustScaler applied

3. Model Evaluation Metrics

Given the severe class imbalance in this dataset, stroke cases represent only 4.9% of all records, accuracy alone is a misleading metric. A model that predicts "no stroke" for every patient would achieve approximately 95% accuracy while being completely useless for the intended purpose. The following metrics were therefore selected to provide a more complete and clinically meaningful evaluation.

Weighted F1-Score:

It was used as the primary metric during cross-validation and hyperparameter tuning. It computes the F1-score for each class separately and takes their weighted average by class support, ensuring that the minority class contributes meaningfully without fully dominating the score.

Stroke Recall:

This is the most clinically important metric in this context. It measures what fraction of actual stroke patients are correctly identified by the model. In a medical screening setting, a false negative, which means failing to identify a patient who will have a stroke, is far more dangerous than a false positive. Maximising this metric is therefore the primary objective.

Stroke Precision and Stroke F1 complement stroke recall:

Stroke precision tracks what fraction of predicted stroke cases are genuine, while stroke F1 provides a single summary that balances the two. These are reported alongside stroke recall to understand the precision-recall trade-off at each operating point.

Accuracy:

It is reported for completeness and comparability with published literature but is not used to drive any modelling decisions due to the reasons described above.

AUC-ROC:

It measures the model's ability to discriminate between the two classes across all possible decision thresholds. It is threshold-independent and provides a useful overall comparison between pipelines regardless of the operating point chosen.

Probability threshold tuning:

This was applied to all pipelines after fitting. Because the dataset is severely imbalanced, the default threshold of 0.5 causes models to predict stroke very rarely. Thresholds of 0.5, 0.4, 0.3, 0.2, and 0.1 were tested on the held-out test set, with the best threshold per pipeline selected by prioritising stroke recall, then stroke F1, then weighted F1.

4. Initial Model Experimentation

To identify the most promising model and preprocessing combinations before committing to full hyperparameter tuning, all five candidate classifiers, mentioned in Deliverable 2, were evaluated using their default hyperparameters across all 16 preprocessed dataset versions. A Stratified K-Fold cross-validation scheme with 5 folds (80% taining and 20% validation) was used, with weighted F1 as the scoring metric. Stratified splitting ensures that the ~4.9% stroke rate is preserved in every fold.

The five classifiers evaluated were Logistic Regression, Random Forest, Gradient Boosting, SVM, and MLP Classifier. A second round of experiments was then conducted with *TruncatedSVD* applied to the training features before fitting, to assess whether dimensionality reduction benefits any model-version combination.

The weighted F1 scores from 5-fold cross-validation are presented in the tables below.

Model	**lg_kn**	**lg_kn_sc**	**lg_kn_sm**	**lg_kn_sm_sc**	**lg_me**	**lg_me_sc**	**lg_me_sm**	**lg_me_sm_sc**
Logistic Regression	0.9276	0.9276	0.9186	0.9189	0.9276	0.9276	0.9263	0.9263
Random Forest	0.9276	0.9276	**0.9709**	**0.9711**	0.9269	0.9269	0.9675	0.9673
Gradient Boosting	**0.9285**	**0.9285**	0.9544	0.9544	0.9273	0.9274	0.9527	0.9527
SVM	0.9276	0.9276	0.7762	0.9311	0.9276	0.9276	0.7761	0.9353
MPL Classifier	0.9276	0.9275	0.9238	0.9552	0.9275	0.9283	0.9254	0.9480

Model	**yj_kn**	**yj_kn_sc**	**yj_kn_sm**	**yj_kn_sm_sc**	**yj _me**	**yj_me_sc**	**yj_me_sm**	**yj_me_sm_sc**
Logistic Regression	0.9276	0.9276	0.9156	0.9174	0.9276	0.9276	0.9191	0.9234
Random Forest	0.9276	0.9276	0.9703	0.9711	0.9278	0.9278	0.9699	0.9695
Gradient Boosting	0.9275	0.9270	0.9506	0.9505	0.9256	0.9264	0.9499	0.9499
SVM	0.9276	0.9276	0.7761	0.9292	0.9276	0.9276	0.7746	0.9309
MPL Classifier	0.9276	0.9280	0.9206	0.9500	0.9276	0.9278	0.9263	0.9422

Model	**Mean**
Logistic Regression	0.9242
Random Forest	**0.9485**
Gradient Boosting	**0.9396**
SVM	0.8906
MPL Classifier	0.9321

The weighted F1 scores from 5-fold cross-validation after SVD dimensionality reduction has been applied to all versions of the dataset are presented in the tables below.

Model	**lg_kn_svd**	**lg_kn_sc _svd**	**lg_kn_sm _svd**	**lg_kn_sm_sc _svd**	**lg_me _svd**	**lg_me_sc _svd**	**lg_me_sm _svd**	**lg_me_sm_sc _svd**
Logistic Regression	0.9276	0.9276	0.7657	0.7010	0.9276	0.9276	0.7705	0.6777
Random Forest	0.9260	0.9278	0.8583	0.7421	0.9263	0.9284	0.8494	0.7177
Gradient Boosting	0.9264	0.9266	0.8189	0.7353	**0.9288**	0.9273	0.8244	0.7114
SVM	0.9276	0.9276	0.7572	0.6965	0.9276	0.9276	0.7584	0.6756
MPL Classifier	0.9276	0.9276	0.7533	0.7109	0.9276	0.9276	0.7542	0.6957

Model	**yj_kn _svd**	**yj_kn_sc _svd**	**yj_kn_sm _svd**	**yj_kn_sm_sc _svd**	**yj _me _svd**	**yj_me_sc _svd**	**yj_me_sm _svd**	**yj_me_sm_sc _svd**
Logistic Regression	0.9276	0.9276	0.7515	0.7034	0.9276	0.9276	0.7524	0.7073
Random Forest	0.9251	0.9259	0.8970	0.7623	0.9264	0.9272	0.9030	0.7694
Gradient Boosting	0.9268	0.9262	0.8334	0.7407	0.9278	0.9256	0.8353	0.7501
SVM	0.9276	0.9276	0.7562	0.7013	0.9276	0.9276	0.7560	0.7068
MPL Classifier	0.9276	0.9276	0.7547	0.7130	0.9275	0.9276	0.7525	0.7209

Model	**Mean**
Logistic Regression	0.8281
Random Forest	0.8695
Gradient Boosting	0.8541
SVM	0.8268
MPL Classifier	0.8297

Observations on the results above:

On the non-SMOTE versions, Gradient Boosting consistently outperformed all other models, with the top two combinations being lg_kn and lg_kn_sc (Logarithmic Transformation + KNN BMI Imputer, with and without RobustScaler), both achieving an F1 of 0.9285. MLP Classifier came close on lg_me_sc (0.9283). Notably, scaling had virtually no effect on non-SMOTE versions for most models, suggesting that the RobustScaler does not meaningfully alter the feature space for tree-based methods when SMOTE has not been applied. Logistic Regression was a weak model overall (mean F1: 0.9242), confirming the expectation that the decision boundary in this dataset is not linear. SVM performed the worst on average (mean F1: 0.8906), largely due to failures on the unscaled SMOTE versions (e.g. lg_kn_sm: 0.7762), where it struggled without normalised input features.

On the SMOTE versions, Random Forest was the clear winner by a wide margin, achieving the three highest scores across the entire experiment: lg_kn_sm_sc (0.9711), lg_kn_sm (0.9709), and yj_kn_sm_sc (0.9704). The gain from SMOTE was most dramatic for Random Forest (mean jump of ~0.043 F1 points), moderate for Gradient Boosting (~0.026), inconsistent for SVM (large gains only when scaling was also applied), and negligible for Logistic Regression. The Logarithmic + KNN imputation combination consistently outperformed its Yeo-Johnson and Mean BMI imputation counterparts across both SMOTE and non-SMOTE settings.

It is important to note a key limitation of these initial results. In this phase, SMOTE was applied to the full training set before cross-validation folds were created. This introduces data leakage, where the created data extracted information from the whole training set, which later got split up into 5 folds, meaning that the validation set was indirectly known, causing the cross-validation scores on SMOTE versions to be optimistically biased. The true generalisation performance of SMOTE-based pipelines is therefore lower than these figures suggest. This limitation is corrected in Section 6, where SMOTE is placed inside the pipeline and applied exclusively within each training fold during cross-validation.

5. Selection of Best-Performing Models and Pre-processed Datasets

Based on the initial experimentation results, two models and five model-version combinations were selected for pipeline construction and hyperparameter tuning. The selection was guided by the cross-validation F1 scores from Section 4, with separate consideration given to SMOTE and non-SMOTE settings, due to the data leakage problem of cross-validating the SMOTE training sets.

Random Forest was selected as the best-performing model on SMOTE versions, achieving the highest F1 scores in the entire experiment. The two versions carried forward are lg_kn_sm_sc (F1: 0.9711) and lg_kn_sm (F1: 0.9709). Both are based on the Logarithmic + KNN imputation combination, which consistently outperformed its Mean BMI and Yeo-Johnson counterparts. Including both the scaled and unscaled variants allows a direct assessment of whether RobustScaler provides any meaningful benefit to Random Forest when SMOTE is present.

Gradient Boosting was selected as the best-performing model on non-SMOTE versions. Three versions are carried forward. lg_kn (F1: 0.9285) and its scaled counterpart lg_kn_sc (F1: 0.9285) were the top-performing non-SMOTE combinations overall. Additionally, lg_me_svd was selected based on the second round of experiments, where this combination produced the best result among all SVD-tested configurations. Including this pipeline allows evaluation of whether compressing the feature space via SVD offers advantages over retaining all features.

Pipeline	**Model**	**Dataset Version**
rf_log_knn_sm_sc	Random Forest	lg_kn_sm_sc
rf_log_knn_sm	Random Forest	lg_kn_sm
gb_log_knn	Gradient Boosting	lg_kn
gb_log_knn_sc	Gradient Boosting	lg_kn_sc
gb_log_mean_svd	Gradient Boosting	lg_me + SVD

SVM and MLP Classifier were not carried forward. SVM underperformed compared to the other models, whereas MLP Classifier showed competitive performance on some scaled SMOTE versions but did not reach the top positions in any category, and its training cost during grid search would be disproportionately high relative to the expected gain.

6. Pipeline Definition

To ensure leak-free cross-validation and reproducible experimentation, all preprocessing steps were integrated with their respective classifiers into end-to-end *Pipeline* objects from the imbalanced-learn library. Using *Pipeline* from *imblearn* rather than *sklearn's* standard *Pipeline* is critical for pipelines that include SMOTE: it ensures that oversampling is applied only to the training fold at each cross-validation split, never to the validation fold, eliminating the data leakage that affected the initial experimentation phase in Section 4.

All preprocessing steps were implemented as sklearn-compatible transformer classes by inheriting from *BaseEstimator* and *TransformerMixin*. This allows *GridSearchCV* to treat the entire pipeline as a single estimator and tune hyperparameters of any step, including the final classifier, in a unified search.

The five pipelines are described below.

1. rf_log_knn_sm_sc (Random Forest + Log + KNN + SMOTE + RobustScaler)

   DataCleaning → LogTransform → KNNImpute → BoolToInt → SMOTE → AddBMIClass → RobustScaler → RandomForest

   This pipeline applies logarithmic transformation to bmi and avg_glucose_level, fills missing BMI values using KNN imputation, converts boolean one-hot columns to integers (required by SMOTE), synthetically oversamples the minority class to a 50/50 balance, adds the ordinal BMI class feature, scales all numerical columns with *RobustScaler*, and finally fits a Random Forest classifier.

2. rf_log_knn_sm (Random Forest + Log + KNN + SMOTE)

   DataCleaning → LogTransform → KNNImpute → BoolToInt → SMOTE → AddBMIClass → RandomForest

   Identical to Pipeline 1 with *RobustScaler* removed. Random Forest is invariant to monotone feature scaling, so this pipeline tests whether the scaler provides any practical benefit when all other steps are identical.

3. gb_log_knn (Gradient Boosting + Log + KNN)

   DataCleaning → LogTransform → KNNImpute → AddBMIClass → GradientBoosting

   A leaner pipeline with no SMOTE and no scaling. This combination was the top-performing non-SMOTE configuration in initial experimentation. Class imbalance is handled implicitly through the model's loss function rather than through resampling.

4. gb_log_knn_sc (Gradient Boosting + Log + KNN + RobustScaler)

   DataCleaning → LogTransform → KNNImpute → AddBMIClass → RobustScaler → GradientBoosting

   Identical to Pipeline 3 with *RobustScaler* added. While Gradient Boosting does not require scaling, this pipeline tests whether normalising the feature ranges influences the model's behaviour in practice, particularly for the gradient computation at early boosting iterations.

5. gb_log_mean_svd (Gradient Boosting + Log + Mean BMI + SVD)

   DataCleaning → LogTransform → MeanBMIImpute → AddBMIClass → SVD (2 components) → GradientBoosting

   This pipeline replaces KNN imputation with Mean BMI Imputer and adds *TruncatedSVD* with 2 components as a dimensionality reduction step before the classifier. SVD compresses the full feature matrix - which is relatively sparse due to one-hot encoding - into a compact dense representation, potentially removing noise and redundant dimensions. This was the best-performing configuration in the SVD experimentation round.

All five pipelines accept the raw original training data (X_train_orig, y_train_orig) as input, with the DataCleaningTransformer as the first step handling all normalisation, type conversion, and one-hot encoding internally. This means the same raw dataframe can be passed to every pipeline without any prior manual preprocessing.

7. Hyperparameter Tuning

GridSearchCV with 5-fold Stratified K-Fold cross-validation (*shuffle= True, random_state= 42*) was applied to each of the five pipelines, using weighted F1 as the scoring metric. Each pipeline was fitted on the raw original training data (X_train_orig, y_train_orig), with all preprocessing - including SMOTE where applicable - executed inside the pipeline at each fold.

For the two Random Forest pipelines (rf_log_knn_sm_sc, rf_log_knn_sm):

Hyperparameter	**Values Searched**
n_estimators	100, 500
max_depth	None, 10
max_features	sqrt
criterion	Gini, entropy
min_samples_split	2, 5
class_weight	None, balanced, balanced_subsample

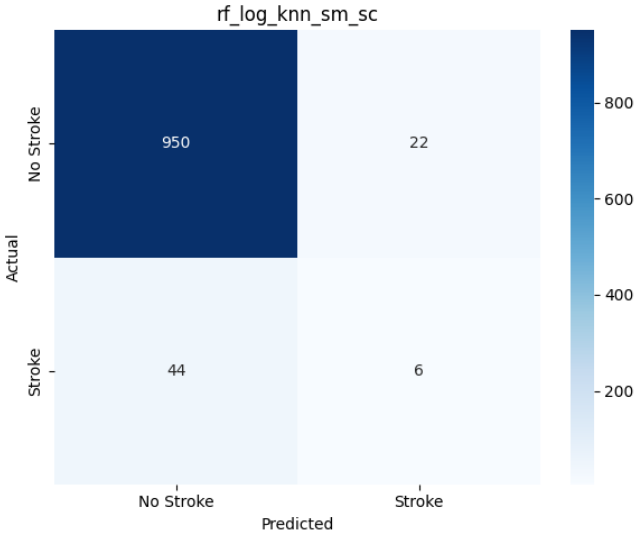
For the three Gradient Boosting pipelines (gb_log_knn, gb_log_knn_sc, gb_log_mean_svd):

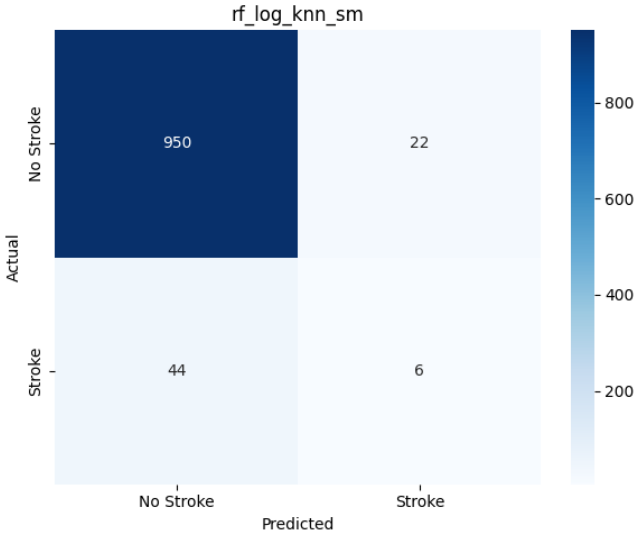
Hyperparameter	**Values Searched**
n_estimators	100, 200, 300
learning_rate	0.01, 0.05, 0.1
max_depth	3, 5
subsample	0.8, 1.0
min_samples_split	2, 5
min_samples_leaf	1, 2

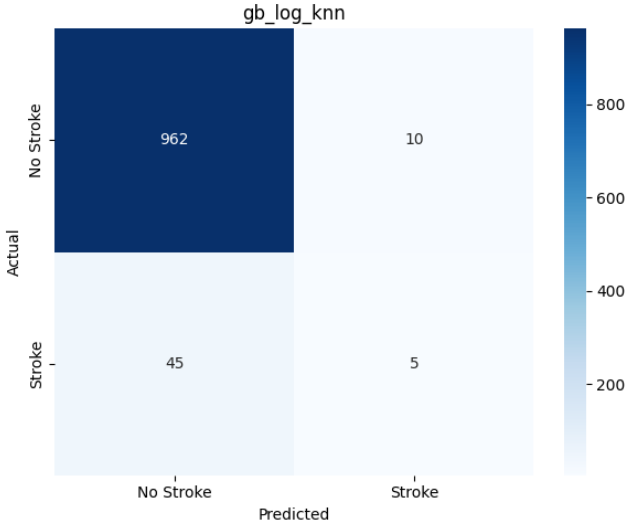
The best cross-validation F1 score, and optimal hyperparameters found for each pipeline are summarised in the tables below.

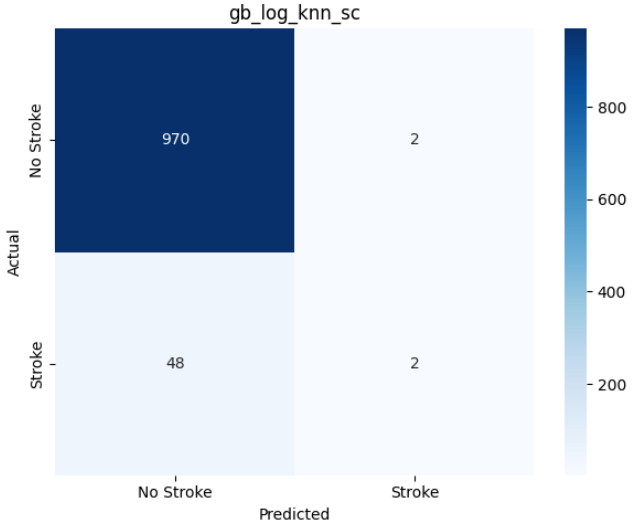
Pipeline	**Best CV F1**	**n_estimators**	**learning_rate**	**max_depth**
rf_log_knn_sm_sc	0.9247	100	-	None
rf_log_knn_sm	0.9252	100	-	None
gb_log_knn	0.9297	100	0.1	5
gb_log_knn_sc	0.9293	200	0.01	5
gb_log_mean_svd	0.9276	100	0.01	3

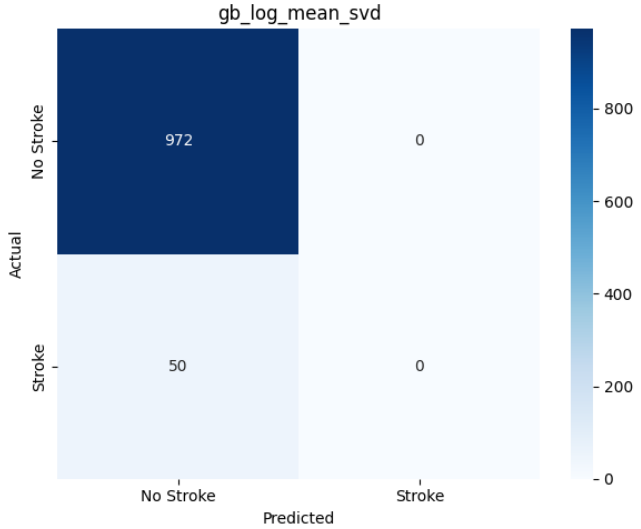
Pipeline	**criterion**	**min_samples_split**	**min_samples_leaf**	**subsample**
rf_log_knn_sm_sc	entropy	2	-	-
rf_log_knn_sm	entropy	2	-	-
gb_log_knn	-	2	2	0.8
gb_log_knn_sc	-	5	2	1.0
gb_log_mean_svd	-	2	1	0.8


rf_log_knn_sm_sc
950
22
44
6
No Stroke
Stroke
Actual
Predicted
800
600
400
200


rf_log_knn_sm
950
22
44
6
No Stroke
Stroke
Actual
Predicted
800
600
400
200


gb_log_knn
962
10
45
5
No Stroke
Stroke
Actual
Predicted
800
600
400
200


gb_log_knn_sc
970
2
48
2
No Stroke
Stroke
Actual
Predicted
800
600
400
200


gb_log_mean_svd
972
0
50
0
No Stroke
Stroke
Actual
Predicted
800
600
400
200
0

Observations on the results above:

1. The Random Forest pipelines achieved lower cross-validation F1 scores (0.9247-0.9252) than the values observed in the initial experimentation phase (0.9709-0.9711). This is expected and is a direct consequence of correcting the data leakage: SMOTE is now applied only within each training fold, so the validation fold always contains only real, unaugmented samples. The pipeline-based scores are therefore more honest estimates of true generalisation performance.

2. The Gradient Boosting pipelines achieved scores (0.9276-0.9297) consistent with their initial experimentation results (~0.9285), confirming that those earlier results were not inflated by leakage since no SMOTE was involved. The best-performing pipeline at this stage is gb_log_knn with a CV F1 of 0.9297, favouring a higher learning rate (0.1), deeper trees (max_depth=5), and subsampling (0.8) to introduce regularising randomness.

3. The optimal class_weight for both Random Forest pipelines was found to be None, meaning no additional class weighting was applied on top of SMOTE. This is consistent with the findings of Experiment 3 reported in Section 8, where class weighting alone (without SMOTE) was found to be ineffective for this dataset.

4. All model-version pairs have a hard time predicting the stroke class. As of now, all models are unusable, and, although their F1 scores are high, the models have less than 0.12 recall.

8. Final model evaluation

8.1 Evaluation at Default Threshold

After hyperparameter tuning, each pipeline was evaluated on the held-out test set of 1,022 samples (972 no-stroke, 50 stroke) using the default decision threshold of 0.5. The results are summarised below.

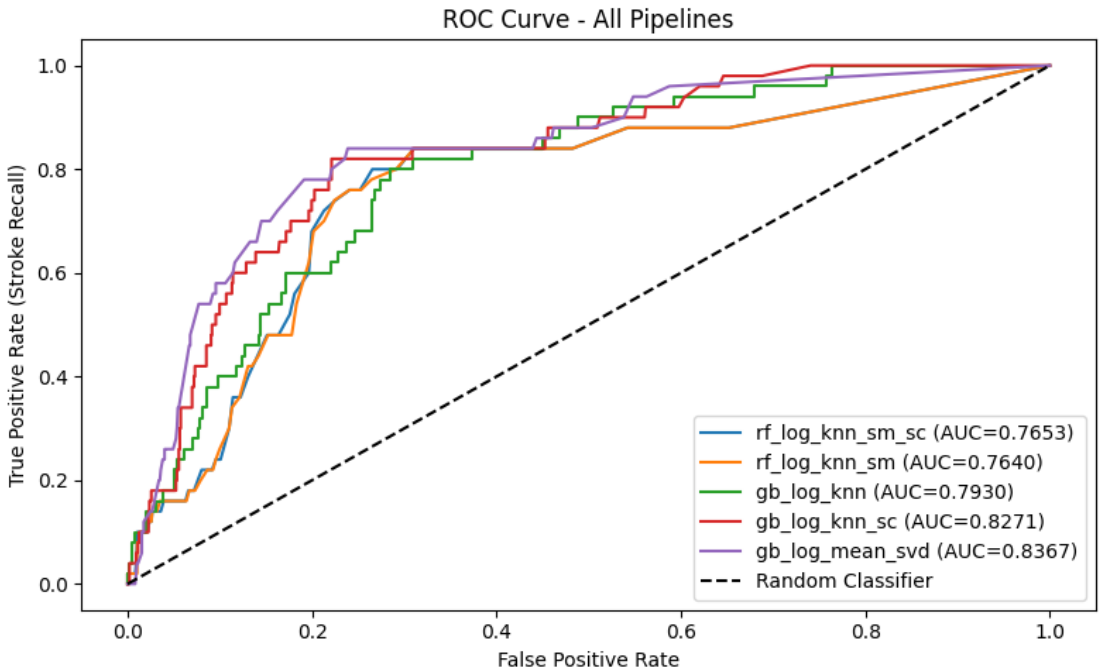
Pipeline	Accuracy	Precision	Recall	F1	Stroke Recall	AUC
gb_log_knn	0.9462	0.9249	0.9462	0.9322	**0.10**	0.7930
gb_log_knn_sc	0.9511	0.9307	0.9511	0.9308	**0.04**	0.8271
gb_log_mean_svd	0.9511	0.9045	0.9511	0.9272	**0.00**	0.8367
rf_log_knn_sm_sc	0.9354	0.9195	0.9354	0.9267	**0.12**	0.7653
rf_log_knn_sm	0.9354	0.9195	0.9354	0.9267	**0.12**	0.7640

The results reveal a fundamental problem with the default threshold. Despite respectable weighted F1 scores (0.9267-0.9322), the Stroke Recall across all pipelines is critically low - ranging from 0.00 to 0.12.

Even the Random Forest pipelines, which include SMOTE, detected only 6 out of 50 stroke cases at threshold 0.5. These results confirm that the default threshold is entirely unsuitable for this task and that threshold tuning is necessary.

The ROC curves show that gb_log_mean_svd achieves the highest AUC (0.8367), followed by gb_log_knn_sc (0.8271) and gb_log_knn (0.7930). The two Random Forest pipelines trail at 0.7653 and 0.7640 respectively, suggesting that despite SMOTE improving training balance, the GB pipelines retain better overall discriminative ability.

<u>ROC curve:</u>



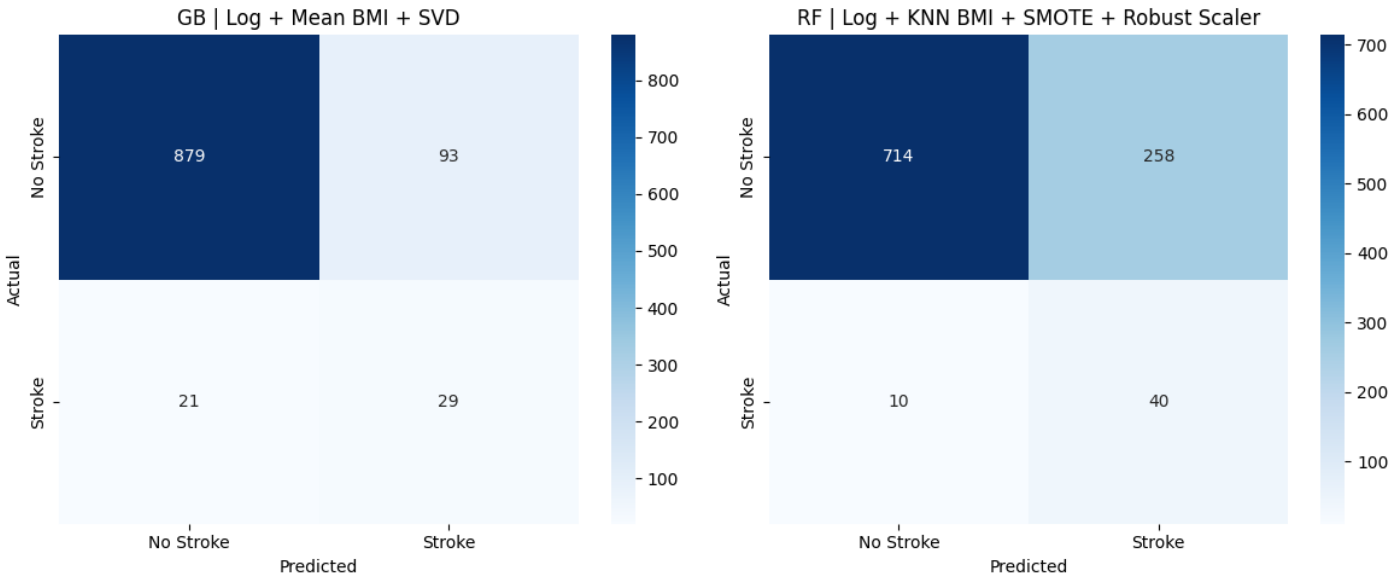
8.2 Threshold Tuning

Since all pipelines failed to detect stroke cases at threshold 0.5, thresholds of 0.4, 0.3, 0.2, and 0.1 were tested for each pipeline. The best threshold per pipeline was selected by prioritising Stroke Recall, then Stroke F1, then weighted F1. For all five pipelines, the optimal threshold was 0.1.

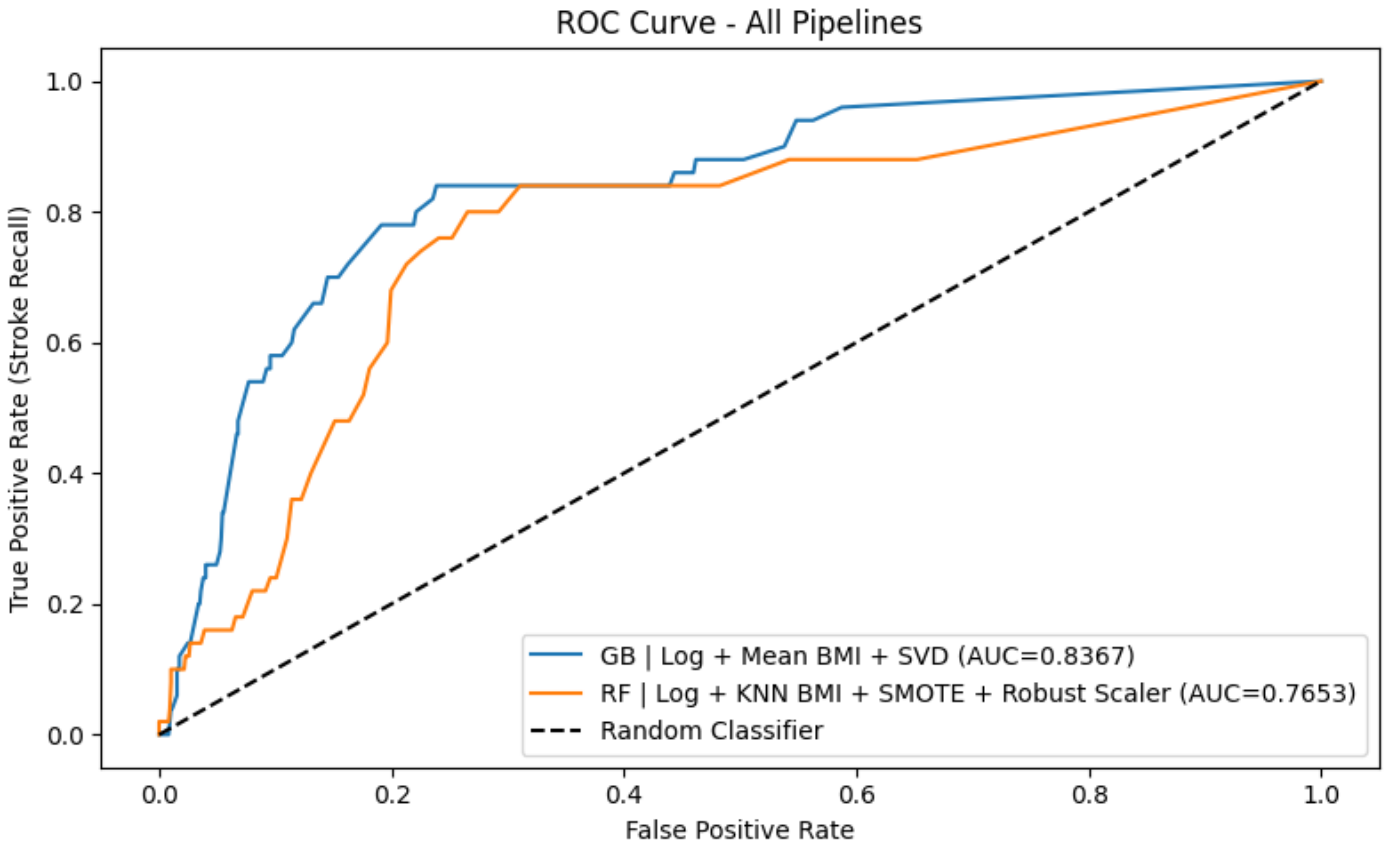
Pipeline	Threshold	Accuracy	F1	Stroke Precision	Stroke Recall	Stroke F1
gb_log_knn	0.1	0.8767	0.898	0.1667	0.38	0.2317
gb_log_knn_sc	0.1	0.8738	0.899	0.2074	0.56	0.3027
gb_log_mean_svd	0.1	**0.8885**	**0.909**	**0.2377**	0.58	**0.3372**
rf_log_knn_sm	0.1	0.7378	0.812	0.1318	0.78	0.2254
rf_log_knn_sm_sc	0.1	0.7378	0.812	0.1342	**0.80**	0.2299

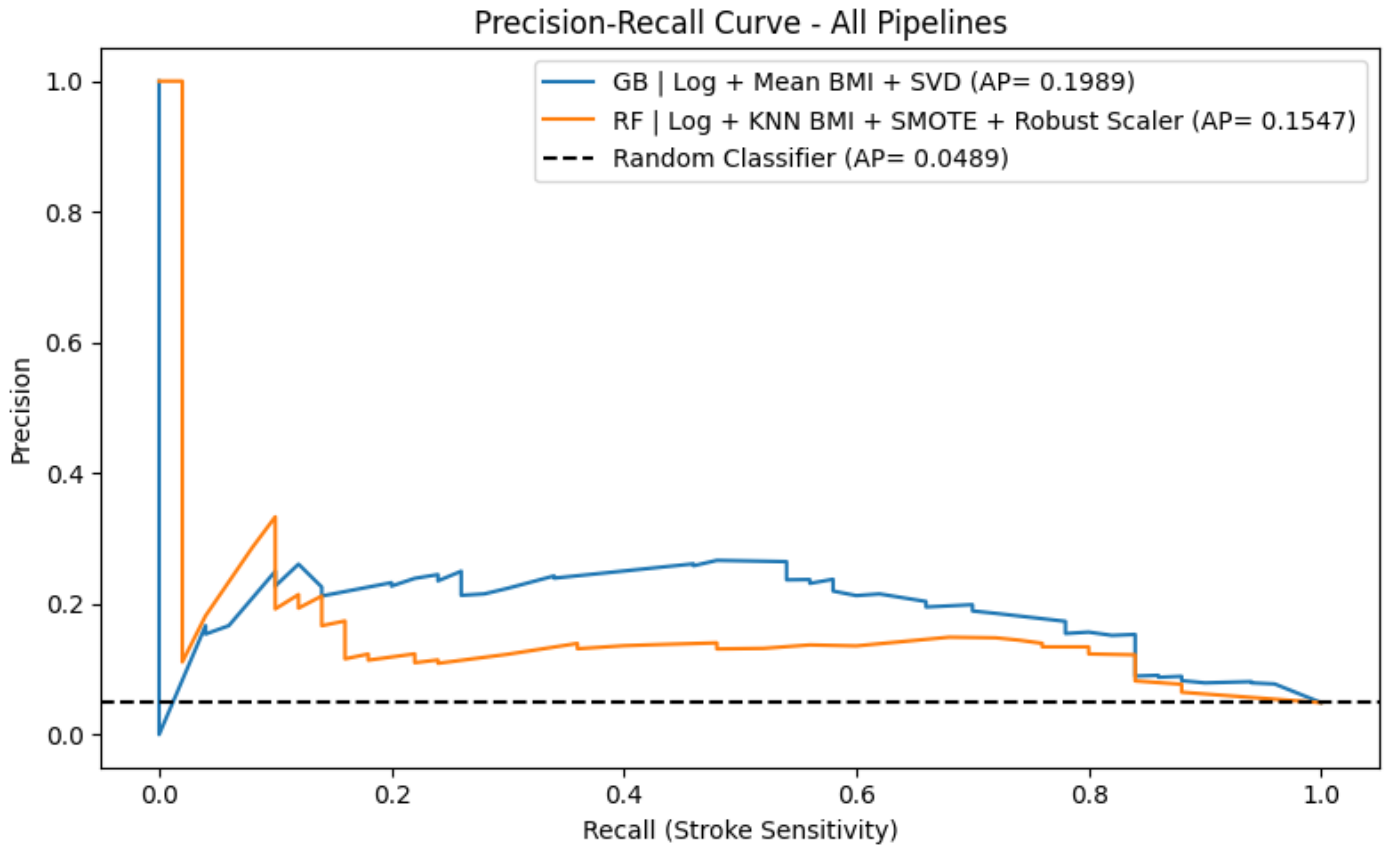
Lowering the threshold to 0.1 dramatically improves stroke detection across all pipelines. Two models stood out:

1. gb_log_mean_svd achieved the highest scores among all others in every metric, except Stroke Recall. It is the best overall model, however, the model is still weak when it comes to correctly predicting the stroke class.
2. rf_log_knn_sm_sc achieved the highest stroke recall, with 40 out of 50 correct stroke classifications.



As shown in the confusion matrices, we can visualise the differences between the two models. Moreover, the Gradient Boosting model is the best performing overall as it is shown from the metrics above, and the following graphs. Gradient Boosting has greater AUC and AP than the Random Forest classifier.


ROC Curve - All Pipelines
True Positive Rate (Stroke Recall)
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
GB | Log + Mean BMI + SVD (AUC=0.8367)
RF | Log + KNN BMI + SMOTE + Robust Scaler (AUC=0.7653)
Random Classifier

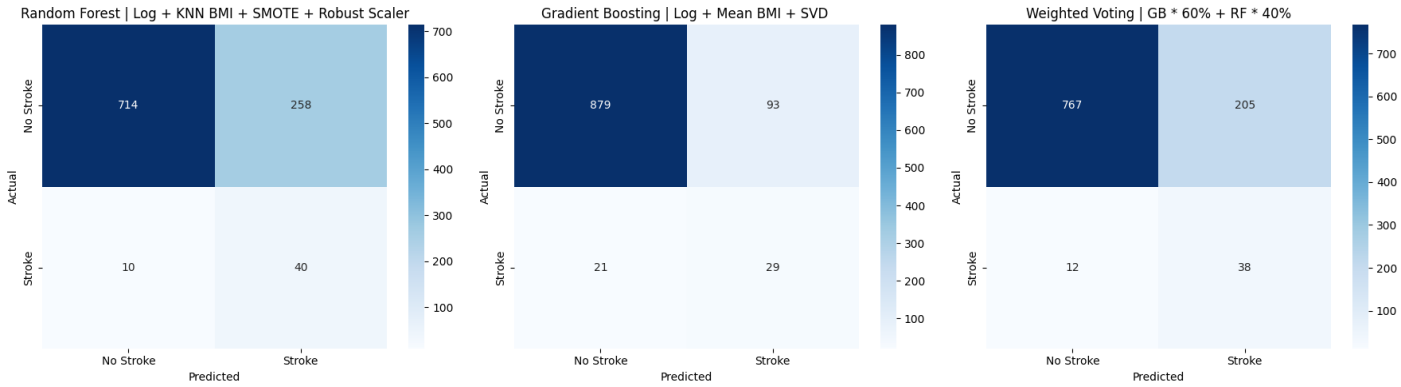

Precision-Recall Curve - All Pipelines
Precision
Recall (Stroke Sensitivity)
0.0
0.2
0.4
0.6
0.8
1.0
GB | Log + Mean BMI + SVD (AP= 0.1989)
RF | Log + KNN BMI + SMOTE + Robust Scaler (AP= 0.1547)
Random Classifier (AP= 0.0489)

8.3 Weighted Voting

The best performance lies between the two discussed models. While keeping the stroke recall high, we aim to minimize the false positives. The prediction of the weighted model is based on the provided probabilities of both models, and then we applied the same 0.1 threshold as before.

new_prediction = (w * GB + (1 - w) * RF) >= 0.1

Weights ranging from 0.2 up to 0.8 with step 0.2 were used to identify the best performing model, which turned out to be 60% GB and 40% RF.



As we can see the Weighted Voting (GB 60% + RF 40%) configuration offers a meaningful trade-off since it reduces false positives to 205 compared to 258 for the standalone RF pipeline, while maintaining a Stroke Recall of 0.76 - catching 38 out of 50 stroke patients. As the higher-weight GB component contributes better-calibrated probabilities, the ensemble benefits from GB's precision while retaining much of RF's sensitivity. This makes it the recommended configuration when both minimising false alarms and maintaining reasonable stroke detection are clinical priorities.

Model	**Stroke Recall**	**False Positives**	**Weighted F1**	**AUC**
GB - Log + Mean BMI + SVD	0.58	**93**	**0.9097**	**0.8367**
RF - Log + KNN BMI + SMOTE + Scale	**0.80**	258	0.8120	0.7653
Weighted Voting (GB 60% + RF 40%)	0.76	205	0.8459	0.7994

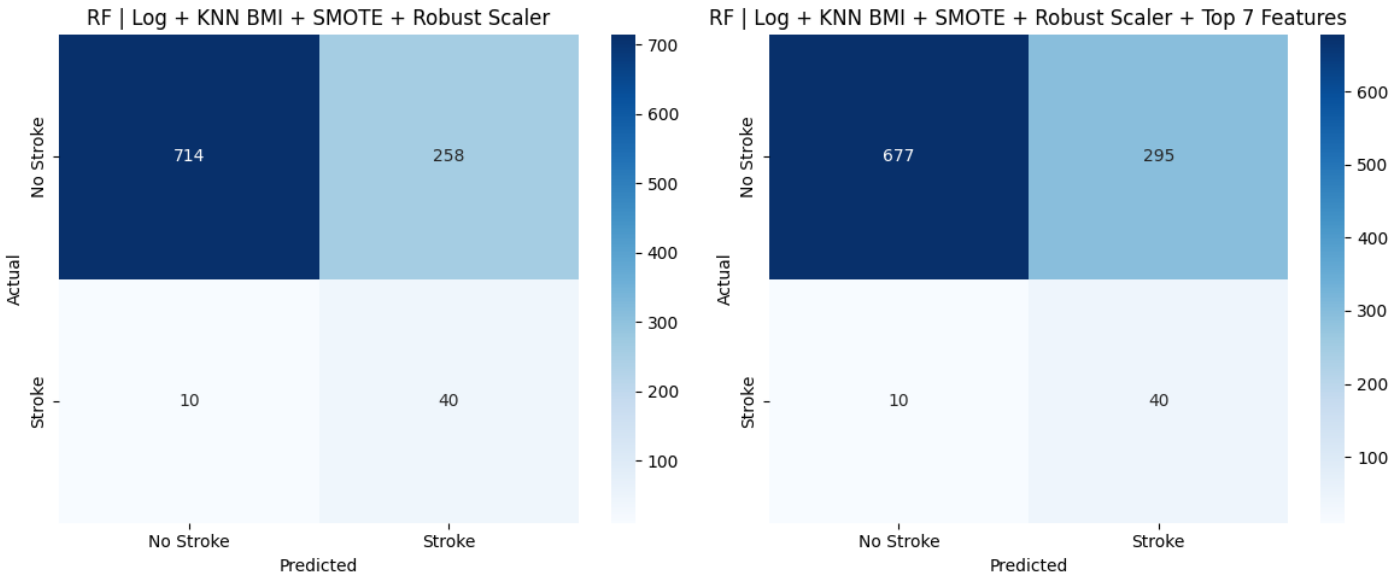
Although the Gradient Boosting still overperforms the other models in terms of metrics, the weighted voting model appears to be the most balanced, while providing a high stroke recall, which is the most vital metric for this scenario.

8.4 Extended Experiments and Final Model Comparison

Following the initial five-pipeline evaluation, three additional experiments were conducted to see whether further gains were achievable.

1. **Feature Selection:**

We trained the best performing models on only the top 3 and top 7 (identified by feature importance) features to check feature selection benefits the classification. A Random Forest trained on only the top 7 features achieved a higher cross-validation F1 (0.9334 vs 0.9247) than the full-feature pipeline, initially suggesting that fewer features might generalise better. However, evaluation on the held-out test set at threshold 0.1 revealed that both models catch the same number of stroke patients (40 out of 50, Stroke Recall = 0.80), while the full-feature pipeline achieves a meaningfully higher AUC (0.8367 vs 0.7828) and fewer false positives (248 vs 295). The higher cross-validation F1 with top 7 features was probably caused by overfitting on the validation folds. This means that even though reducing features lowered variance during cross-validation the generalisation hurt. Therefore, all original features are retained.



2. **SMOTE Ratio:**

The SMOTE strategy used was to create synthetic data until the smaller class reached the number of rows as the other class. We checked whether other ratios such as 70/30 and 65/35 should yield better results. The F1 score of the other ratios were 0.02 more or less than the 50/50 ratio used throughtout the work, so we did not further test this experiment.

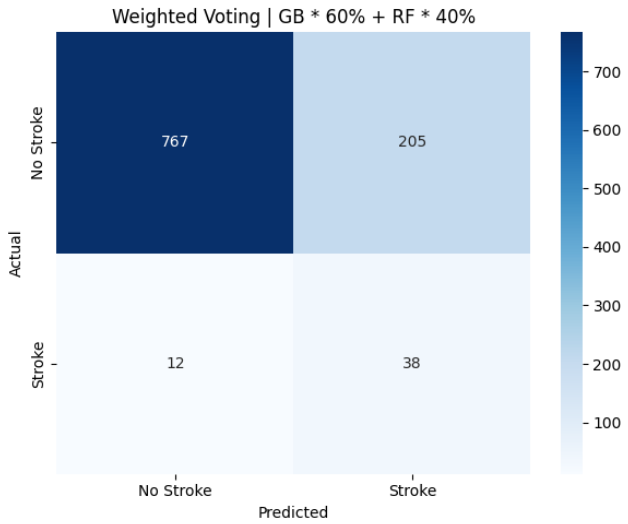
3. **SMOTEENN Pipeline:**

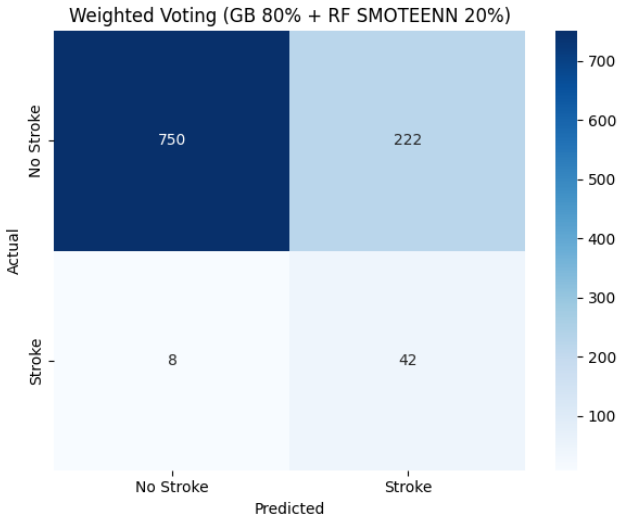
Beyond the initial comparison of SMOTE variants, a complete tuned pipeline equivalent to rf_log_knn_sm_sc was built with SMOTEENN replacing SMOTE (rf_log_knn_smoteenn_sc) and subjected to the same GridSearchCV procedure. At threshold 0.1 on the test set, this pipeline achieved a Stroke Recall of 0.84 (42 out of 50 stroke patients detected) with AUC = 0.8084, outperforming the standard SMOTE pipeline on both metrics. However, it produced significantly more false positives (331 vs 258), meaning more healthy patients would be unnecessarily flagged for further clinical testing.

The better performance prompted us to try a three-way weighted voting system between the two best performing models already identified, and the SMOTEENN version.

new_prediction = (w1 * GB + w2 * RF + w3 * RF_SMOTEENN) >= 0.1, w1 + w2 + w3 = 1

The best weight distribution turned out to be w1= 0.8, w2= 0, and w3= 0.2, meaning the best weighted voting in terms of stroke recall and then F1 score, preferred the Random Forest SMOTEENN model over the Random Forest classifier on the SMOTE dataset.





The SMOTEENN weighted voting model has greater stroke recall, but slightly more false positives. It looks like the better model overall out of all models identified.

9. Discussion

9.1 Key Findings

The central finding of this study is that two distinct operating modes emerge across the five pipelines, representing different clinical priorities.

The rf_log_knn_sm_sc pipeline, at threshold 0.1, achieves the highest Stroke Recall of 0.80 - detecting 40 out of 50 stroke patients in the test set. This makes it the preferred choice for a high-sensitivity screening tool where missing a stroke patient carries a greater cost than a false alarm. However, this performance comes at a significant cost to precision: of the 298 patients flagged as stroke-positive, only 40 are genuine cases (Stroke Precision = 0.1342). The low AUC (0.7653) suggests that while SMOTE helps the model detect more stroke cases, it distorts the model's confidence estimates.

The gb_log_mean_svd pipeline represents the best overall performer across all probabilistic metrics. It achieves the highest AUC (0.8367), the highest Average Precision (0.1989), the best weighted F1 at threshold 0.1 (0.9097), and the best Stroke F1 (0.3372). Its stroke recall of 0.58 at threshold 0.1 - identifying 29 out of 50 stroke patients - is lower than the RF pipelines, but it does so with meaningfully better precision (0.2377) and far fewer false alarms (93 vs 258). The SVD dimensionality reduction step appears to benefit this pipeline by compressing the sparse one-hot encoded feature space, reducing noise and producing better-calibrated probability estimates.

Feature selection did not improve generalisation performance. Although reducing to the top 7 features produced a higher cross-validation F1 (0.9334 vs 0.9247), evaluation on the test set revealed that the full-feature pipeline achieves better AUC (0.8367 vs 0.7828) and fewer false positives (248 vs 295) with the same Stroke Recall. The apparent gain during cross-validation was caused by variance reduction on the folds rather than genuine improvement.

Among the SMOTE variants tested, SMOTEENN produced the best minority-class metrics in the standalone experiment. When built into a full tuned pipeline (rf_log_knn_smoteenn_sc), it achieved the highest Stroke Recall of any single model (0.84, catching 42 out of 50 stroke patients) with AUC = 0.8084, but at the cost of 331 false positives - more than any other configuration. The Weighted Voting approach (GB 80% + RF SMOTEEN 20%) offers the most practical trade-off, reducing false positives to 222 while maintaining a Stroke Recall of 0.84 - catching 42 out of 50 stroke patients. This makes it the recommended configuration when both minimising false alarms and maintaining reasonable stroke detection are clinical priorities.

9.2 Comparison with Literature

Study	Algorithm	Handling Imbalance	Accuracy	AUC	Notes
This study (best recall)	SMOTEENN weighted voting	SMOTEEN	0.7755	0.8278	Stroke Recall = 0.84, threshold = 0.1
This study (best overall)	Gradient Boosting + SVD	None	0.9511	0.8367	Stroke Recall = 0.58, threshold = 0.1
Emon et al. (2020) [1]	Random Forest	None	0.9600	-	High accuracy, imbalance not addressed
Dev et al. (2022) [2]	Neural Network	SMOTE	-	0.8600	Focus on neural architectures
Dritsas & Trigka (2022) [3]	Stacking Ensemble	Oversampling	**0.9800**	**0.9900**	Ensemble of multiple classifiers
Singhai et al. (2024) [4]	XGBoost	SMOTE	0.9500	0.8800	Similar feature set

Several studies in the literature report very high accuracy values (0.96-0.98), but these are largely a reflection of the class imbalance rather than genuine predictive power - predicting "no stroke" for every patient already achieves ~95% accuracy on this dataset. Studies that report AUC and minority-class metrics show values more consistent with ours. The gb_log_mean_svd pipeline's AUC of 0.8367 is competitive with the published literature.

9.3 Domain-Specific Validation

The feature importance analysis from the Random Forest classifier identified age, bmi, and avg_glucose_level as the three most important predictors, which is strongly consistent with established clinical knowledge. Stroke risk is well-documented to increase significantly with age, and elevated average glucose levels are a recognised indicator of diabetes - itself a major independent risk factor for stroke. The elevated rates of hypertension (26.5% among stroke cases vs 9.8% overall) and heart disease (18.8% vs 5.4% overall) in stroke-positive patients are likewise consistent with decades of epidemiological evidence. The fact that the models converge on these clinically meaningful features without any domain guidance gives the results face validity from a medical perspective.

The finding that residence type showed negligible predictive importance is also medically plausible - while socioeconomic factors do influence stroke risk, the urban/rural binary in this dataset is too coarse to capture meaningful health inequality signals.

9.4 Study Limitations

1. The dataset is severely imbalanced with only 249 stroke cases out of 5,110 records (~4.9%), which constrains the reliability of minority-class metrics - small changes in the number of true positives (e.g. detecting 39 vs 40 out of 50 stroke patients) produce large swings in Stroke Recall.

2. Threshold tuning was performed on the test set rather than a dedicated validation set, introducing a mild optimistic bias in the reported threshold-dependent metrics.

3. The smoking_status "Unknown" category accounts for approximately 30% of all records and is treated as a valid category throughout; this represents a substantial source of uncertainty that may reduce model generalisability.

4. The gb_log_mean_svd pipeline uses only 2 SVD components, which was chosen based on experimentation rather than principled component selection - a more systematic approach such as explained variance analysis could improve this choice.

5. SMOTEENN was incorporated into a full tuned pipeline and, when combining with the GB model in weighted voting, demonstrated the highest Stroke Recall (0.84). However, it produced many false positives (222 out of 972 ~23%), which represents a real clinical cost - healthy patients incorrectly flagged for further testing. Future work could explore threshold optimisation and cost-sensitive evaluation to better quantify this trade-off for specific clinical deployment scenarios.

10. References

[1] M. U. Emon, M. S. Keya, T. I. Meghla, M. M. Rahman, M. S. A. Mamun, and M. S. Kaiser, “Performance analysis of machine learning approaches in stroke prediction,” in Proc. 2020 4th Int. Conf. Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, Nov. 2020, pp. 1464-1469, doi: 10.1109/ICECA49313.2020.9297525.

[2] S. Dev, H. Wang, C. S. Nwosu, N. Jain, B. Veeravalli, and D. John, “A predictive analytics approach for stroke prediction using machine learning and neural networks,” Healthcare Analytics, vol. 2, art. no. 100032, Nov. 2022, doi: 10.1016/j.health.2022.100032.

[3] E. Dritsas and M. Trigka, “Stroke Risk Prediction with Machine Learning Techniques,” Sensors, vol. 22, no. 13, Art. no. 4670, 2022, doi: 10.3390/s22134670.

[4] H. K. Singhai, P. K. Shukla, and V. S. Dhaka, “An experimental analysis of brain stroke prediction using machine learning algorithms,” in *Proc. 2024 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, Gwalior, India, 2024, pp. 1-6, doi: 10.1109/IATMSI60426.2024.10503299.